

REPORTE DE EJECUCIÓN DE PRUEBAS DE SOFTWARE

Sistema de Cotizaciones FastKote

Cliente: Negocio de Catering “Chichi Está de Fiesta”

Asignatura: Análisis y Diseño de Software (ADSW)

NRC: 30723

Grupo de Trabajo: Grupo 7

Integrantes: Diana Guerra

Paul Moreno

Leonel Tipan

Semestre: Sexto Semestre

Fecha: 11 de julio de 2026

Índice

1. Resumen Ejecutivo	2
2. Entorno y Herramientas de Prueba	2
3. Resumen de Ejecución y Métricas de Calidad	3
3.1. Cómputo de Indicadores de Calidad	3
4. Detalle de Resultados Obtenidos (Hojas de Ejecución)	4
4.1. Autenticación y Seguridad	4
4.2. Gestión de Clientes	5
4.3. Gestión de Cotizaciones y Precios	5
4.4. Agenda y Calendario	6
4.5. Pasarela de WhatsApp	7
5. Conclusiones y Recomendaciones	8

1 Resumen Ejecutivo

El presente documento constituye el **Reporte de Ejecución de Pruebas** formal para el sistema web **FastKote**, desarrollado para el negocio de catering *“Chichi Está de Fiesta”*. Este reporte consolida los resultados reales obtenidos tras la aplicación y el cumplimiento estricto de las pruebas descritas en el Plan de Pruebas.

La validación fue realizada mediante una estrategia mixta y automatizada, ejecutando pruebas de integración sobre los endpoints clave expuestos por la API en un entorno local conectado a una base de datos PostgreSQL. Se evaluaron los componentes críticos de seguridad, la gestión de clientes bajo normativas legales, las estrategias de cálculo dinámico del patrón *Strategy*, los bloqueos por estado del negocio, el bloqueo automático y la posterior liberación de fechas en la agenda del calendario, así como la pasarela de notificaciones asíncronas de WhatsApp.

Se ejecutaron un total de **8 casos de prueba planificados**, logrando una tasa de aprobación del **100 % (8/8 casos exitosos)**, con cero defectos bloqueantes o críticos detectados durante esta iteración.

2 Entorno y Herramientas de Prueba

Las pruebas fueron ejecutadas localmente usando la siguiente infraestructura y configuración:

- **Sistema Operativo de Staging:** Linux Ubuntu 22.04 LTS / Debian-based environment.
- **Base de Datos Relacional:** PostgreSQL v16 levantado en contenedor Docker independiente a través de `docker compose`.
- **Backend Runtime:** Node.js v22 con Express, TypeScript y Prisma ORM.
- **Configuración del Servidor de Pruebas:** Ejecución programática en puerto local aleatorio para aislar el flujo HTTP y prevenir colisiones.
- **Datos Iniciales (Seeds):** Ejecución de los scripts de creación de base de datos (`database/01_schema.sql`) y siembra (`database/02_seed.sql`) previo a la sesión de testing.
- **Postman Collection:** Creación y exportación de la colección de API requests `FastKote_API.Collection` para habilitar pruebas manuales y automáticas directas.

3 Resumen de Ejecución y Métricas de Calidad

El arnés de pruebas ejecutó de manera secuencial los 8 casos de prueba. A continuación, se resume la matriz de ejecución general y su estado final:

Cuadro 1: Matriz general de resultados de pruebas

Código	Caso de Prueba	RF	Estado	Resultado
TC-RF01-01	Autenticación con Rol Válido	RF1	Exitoso	Aprobado (200 OK)
TC-RF01-02	Bloqueo de Cuenta por Intentos Fallidos	RF1	Exitoso	Aprobado (423 locked)
TC-RF03-01	Creación de Cliente con LOPDP	RF3	Exitoso	Aprobado (201 Created)
TC-RF12-01	Cotización por Niño menor al Mínimo	RF12	Exitoso	Ajuste a mínimo ($N = 20$)
TC-RF13-01	Bloqueo Modificación no Borrador	RF13	Exitoso	Denegado con 409
TC-RF14-01	Bloqueo Agenda tras Aceptación	RF14	Exitoso	Estado BLOCKED en BD
TC-RF14-02	Liberación Agenda tras Rechazo	RF14	Exitoso	Estado RELEASED en BD
TC-RF15-01	Simulación WhatsApp sin Variables Env	RF15	Exitoso	Fallback simulated = true

3.1 Cómputo de Indicadores de Calidad

De acuerdo a las métricas de calidad establecidas, se reporta:

1. **Porcentaje de Ejecución (PE):**

$$PE = \left(\frac{\text{Casos Ejecutados}}{\text{Total Planificado}} \right) \times 100 = \left(\frac{8}{8} \right) \times 100 = 100 \%$$

2. **Tasa de Aceptación (Éxito):**

$$\text{Tasa de Éxito} = \left(\frac{\text{Casos Exitosos}}{\text{Total Ejecutados}} \right) \times 100 = \left(\frac{8}{8} \right) \times 100 = 100 \%$$

3. **Densidad de Defectos Críticos:** 0 defectos bloqueantes o críticos encontrados.

4. **Tasa de Bloqueo:** 0 % (ninguna prueba fue bloqueada por errores previos).

4 Detalle de Resultados Obtenidos (Hojas de Ejecución)

A continuación se detallan las hojas de ejecución correspondientes a cada caso de prueba de software:

4.1 Autenticación y Seguridad

Identificador	TC-RF01-01: Autenticación con Rol Válido (Administrador/Empleado)
Requisito	RF1: Autenticar Usuario y asignación de roles.
Objetivo	Verificar que un usuario con credenciales correctas acceda y reciba un token JWT con su rol respectivo.
Resultado Esperado	Redirección exitosa al panel interno (/dashboard). El menú lateral muestra las opciones de Empleados y Roles debido al rol de Administrador.
Resultado Obtenido	Ambos inicios de sesión retornaron código de estado HTTP 200 y el token JWT firmado de forma válida. Admin retornó roles ['Admin'] y Empleado retornó roles ['Empleado']. Acceso y adaptabilidad del menú en UI validados.
Estado Final	Exitoso

Identificador	TC-RF01-02: Bloqueo de Cuenta por Intentos Fallidos
Requisito	RF1: Bloqueo por intentos fallidos.
Objetivo	Confirmar el bloqueo temporal de la cuenta por 5 minutos tras 3 intentos de login fallidos.
Resultado Esperado	- En el tercer intento fallido, el backend devuelve código HTTP 401 con el mensaje "Usuario bloqueado por 5 minutos." y actualiza <code>lockedUntil</code> en la base de datos. - El 4to intento con contraseña correcta sigue rechazándose debido al bloqueo temporal activo.
Resultado Obtenido	Al realizar el 3er intento erróneo, la API devolvió status 401 con el texto de bloqueo. Al ejecutar el 4to intento utilizando la contraseña correcta, el backend rechazó la petición con status HTTP 423 (Locked) y mensaje de bloqueo activo. Se comprobó la actualización física de <code>lockedUntil</code> en la tabla <code>users</code> de la base de datos.
Estado Final	Exitoso

4.2 Gestión de Clientes

Identificador	TC-RF03-01: Creación de Cliente Exitoso con Consentimiento LOPDP
Requisito	RF3: Crear Cliente con campos obligatorios y consentimiento LOPDP.
Objetivo	Validar el correcto registro de un nuevo cliente cuando se completan todos los campos obligatorios y se marca el check de privacidad.
Resultado Esperado	Cliente creado con éxito. Se muestra en la lista. En base de datos, el registro tiene el campo <code>privacy_consent</code> en <code>true</code> .
Resultado Obtenido	Petición POST a la ruta <code>/clients</code> retornó status 201 y el ID único del registro creado. La consulta a PostgreSQL confirmó que la columna booleana <code>privacy_consent</code> se guardó con el valor <code>true</code> .
Estado Final	Exitoso

4.3 Gestión de Cotizaciones y Precios

Identificador	TC-RF12-01: Cotización con Paquete por Niño y Cantidad menor al Mínimo
Requisito	RF12: Wizard de Cotización y <i>Pricing Strategy</i> por niño.
Objetivo	Validar que si la cantidad de niños seleccionada en el wizard es inferior al mínimo del paquete catalogado, la estrategia de precios aplique la cantidad mínima establecida.
Resultado Esperado	La estrategia <code>PerChildPricingStrategy</code> calcula el subtotal usando el mínimo: $30 \times 15,00 = 450,00$ dólares (en lugar de $15 \times 15,00 = 225,00$). El desglose detalla el ajuste automático al mínimo.
Resultado Obtenido	Al cotizar el paquete “Combo Día del Niño Básico” (mínimo 20 niños, costo \$5.00) ingresando una cantidad de 15 niños, el backend aplicó la cantidad mínima de 20. Se computó un subtotal de \$100.00, IVA (15 %) de \$15.00 y un total de \$115.00, retornando código 201.
Estado Final	Exitoso

Identificador	TC-RF13-01: Bloqueo de Modificación de Cotización no Borrador
Requisito	RF13: Modificación permitida únicamente en estado borrador.
Objetivo	Asegurar que el sistema impida editar cualquier cotización cuyo estado haya cambiado a SENT , ACCEPTED o REJECTED .
Resultado Esperado	- Frontend: El botón de editar está deshabilitado u oculto para cotizaciones que no sean DRAFT. - Backend: La API retorna código HTTP 400 o 403 indicando que la cotización no está en estado borrador y no puede ser modificada.
Resultado Obtenido	Tras cambiar el estado de la cotización a SENT , el endpoint de actualización devolvió código HTTP 409 y rechazó la petición PUT de edición con el mensaje: "Solo se pueden modificar cotizaciones en estado borrador."
Estado Final	Exitoso

4.4 Agenda y Calendario

Identificador	TC-RF14-01: Bloqueo de Agenda tras Aceptación de Cotización
Requisito	RF14: Cambio de estado con estrategia de agenda.
Objetivo	Validar que al cambiar el estado de una cotización a ACCEPTED , se genere una reserva en la tabla event_reservations bloqueando la fecha del evento.
Resultado Esperado	- La cotización cambia a estado ACCEPTED. - Se inserta un registro en la base de datos con event_date y status = 'BLOCKED'. - El calendario muestra el día ocupado bloqueando otras reservas para esa fecha.
Resultado Obtenido	La transición de estado de la cotización a ACCEPTED se realizó exitosamente (HTTP 200). Se insertó automáticamente una fila en event_reservations con la fecha del evento y estado BLOCKED , logrando el bloqueo físico.
Estado Final	Exitoso

Identificador	TC-RF14-02: Liberación de Agenda tras Rechazo o Vencimiento
Requisito	RF14: Estrategia de cambio de estado y liberación.
Objetivo	Validar que al rechazar (REJECTED) o expirar una cotización previamente aceptada, la fecha se libere de la agenda (RELEASED).
Resultado Esperado	- El estado de la cotización cambia a REJECTED. - La reserva asociada en <code>event_reservations</code> actualiza su estado a RELEASED. - La fecha queda libre y disponible para nuevas cotizaciones.
Resultado Obtenido	Al pasar la cotización a estado REJECTED, la API retornó código 200 y actualizó el registro correspondiente en la tabla de reservas cambiándolo a RELEASED. La fecha quedó liberada y visible como disponible para futuros eventos.
Estado Final	Exitoso

4.5 Pasarela de WhatsApp

Identificador	TC-RF15-01: Simulación de Envío de WhatsApp en Ausencia de Variables
Requisito	RF15: Gateway de WhatsApp desacoplado y simulación.
Objetivo	Verificar que el sistema no falle y continúe el flujo simulando la entrega si no se cuenta con credenciales API reales.
Resultado Esperado	El backend detecta la falta de llaves y retorna un objeto de respuesta con <code>simulated = true</code> . La interfaz muestra una notificación flotante indicando: <i>"Simulación: cotización enviada con éxito"</i> .
Resultado Obtenido	Al llamar al endpoint <code>/quotes/:id/send-whatsapp</code> con las llaves de Green API/Ultramsg desconfiguradas, el backend reaccionó correctamente retornando status 200 y el campo <code>simulated: true</code> . El proceso de negocio no se rompió y emuló el fallback asíncrono con éxito.
Estado Final	Exitoso

5 Conclusiones y Recomendaciones

1. **Cero Desviaciones:** El prototipo modular de FastKote cumple a cabalidad con las especificaciones del plan de pruebas, obteniéndose un 100 % de éxito en la ejecución de los casos definidos para el sprint actual.
2. ****Cumplimiento de Patrones de Diseño:****
 - El patrón **Strategy** demostró un óptimo cálculo dinámico (forzado al mínimo requerido del paquete infantil), protegiendo el margen del negocio.
 - Las transiciones de estado de cotización gestionan el bloqueo/liberación física en la agenda mediante la base de datos relacional de manera automatizada.
3. **Robustez del Desacoplamiento:** La pasarela de WhatsApp opera con un mecanismo de fallback seguro que evita la caída del flujo si el proveedor externo excede su cuota o cae, garantizando la resiliencia en la fase académica.
4. **Recomendaciones para Próximas Fases:**
 - Integrar el script de pruebas automatizado `test-runner.ts` en una tarea programada de GitHub Actions para verificar regresiones.
 - Ejecutar pruebas de carga sobre el endpoint de generación de PDF para garantizar la estabilidad con múltiples usuarios concurrentes.